

API Documentation

API Documentation

This document provides comprehensive documentation for all API endpoints in the Connecting Future platform, including serv

Base URLs:

- Backend API: `http://localhost:4000` (development)
- Frontend API: `http://localhost:3000/api` (Next.js API routes)

Authentication:

- Many backend routes use JWT authentication via Auth0 (`checkJwt` middleware)
- Protected endpoints require `Authorization: Bearer <token>` header
- Authentication status is indicated for each endpoint

Table of Contents

1. [Data Models](#data-models)
2. [Auth & User Endpoints](#auth--user)
3. [Academic Progress Endpoints](#academic-progress)
4. [Donations Endpoints](#donations)
5. [Roadmaps Endpoints](#roadmaps)
6. [Jobs & Applications](#jobs--applications)
7. [Messaging Endpoints](#messaging)
8. [Connections Endpoints](#connections-network)
9. [Mentorship Endpoints](#mentorship-mentors-requests-sessions-ratings)
10. [Frontend Next.js API Routes](#frontend-nextjs-api-routes-proxies--payment-handlers)

Data Models

User Model

```
``json
{
  "auth0_id": "string",
  "email": "string",
  "name": "string",
  "picture": "string (URL)",
  "role": "student | alumni | admin",
  "graduationYear": "number (optional)",
  "course": "string (optional)",
  "currentCompany": "string (optional)",
  "jobTitle": "string (optional)",
  "location": "string (optional)",
  "linkedin": "string (URL, optional)",
  "bio": "string (optional)",
  "skills": "string (comma-separated, optional)",
  "isOpenToMentoring": "boolean (optional)",
  "created_at": "timestamp",
  "updated_at": "timestamp"
}
```

Semester Model

```
``json
{
  "semester_key": "string (unique identifier)",
  "student_auth0_id": "string",
```

```

"name": "string (e.g., 'Semester 7')",
"gpa": "number (0-10)",
"total_credits": "number",
"is_current": "boolean",
"courses": "Course[] (array of course objects)"
}
...

```

Course Model

```

```json
{
 "course_key": "string (unique identifier)",
 "student_auth0_id": "string",
 "semester_key": "string",
 "name": "string",
 "code": "string (e.g., 'CS101')",
 "credits": "number",
 "grade": "string (e.g., 'A', 'B+')",
 "status": "upcoming | in-progress | completed",
 "progress": "string (optional)"
}
...

```

#### ### Donation Model

```

```json
{
  "id": "number (auto-increment)",
  "donor_name": "string",
  "donor_email": "string",
  "amount": "number (decimal)",
  "currency": "string (default: 'INR')",
  "payment_method": "razorpay | cash | bank_transfer",
  "transaction_status": "pending | completed | failed",
  "transaction_id": "string (optional)",
  "razorpay_order_id": "string (optional)",
  "razorpay_payment_id": "string (optional)",
  "razorpay_signature": "string (optional)",
  "donation_date": "timestamp",
  "receipt_sent": "boolean",
  "receipt_url": "string (optional)",
  "notes": "text (optional)"
}
...

```

Job Model

```

```json
{
 "id": "number (auto-increment)",
 "title": "string",
 "company": "string",
 "description": "text",
 "requirements": "text",
 "job_type": "full-time | part-time | internship | contract",
 "industry": "string",
 "location": "string",
 "remote": "boolean",
 "salary_min": "number (optional)",
 "salary_max": "number (optional)",
 "application_deadline": "date (optional)",
 "posted_by": "string (email)",
 "status": "active | closed | draft",
 "created_at": "timestamp",
 "updated_at": "timestamp"
}
...

```

```
}
...
```

#### ### Application Model

```
```json  
{  
  "id": "number (auto-increment)",  
  "job_id": "number",  
  "applicant_email": "string",  
  "resume_url": "string",  
  "cover_letter": "text (optional)",  
  "status": "applied | withdrawn | accepted | rejected",  
  "applied_at": "timestamp",  
  "updated_at": "timestamp"  
}  
...
```

Message Model

```
```json  
{
 "id": "number (auto-increment)",
 "sender_email": "string",
 "receiver_email": "string",
 "content": "text (encrypted)",
 "is_read": "boolean",
 "created_at": "timestamp"
}
...
```

#### ### Connection Model

```
```json  
{  
  "id": "number (auto-increment)",  
  "requester_email": "string",  
  "target_email": "string",  
  "status": "pending | accepted | rejected | removed",  
  "message": "text (optional)",  
  "created_at": "timestamp",  
  "responded_at": "timestamp (optional)"  
}  
...
```

Mentor Profile Model

```
```json  
{
 "mentor_email": "string (primary key)",
 "expertise_areas": "string (comma-separated)",
 "years_experience": "number",
 "hourly_rate": "number (decimal, optional)",
 "availability": "string",
 "bio": "text",
 "rating": "number (1-5, computed)",
 "total_sessions": "number",
 "total_ratings": "number",
 "created_at": "timestamp",
 "updated_at": "timestamp"
}
...
```

#### ### Mentorship Request Model

```
```json  
{  
  "id": "number (auto-increment)",
```

```

"student_email": "string",
"mentor_email": "string",
"message": "text",
"status": "pending | accepted | rejected",
"created_at": "timestamp",
"responded_at": "timestamp (optional)"
}
...

```

Mentorship Session Model

```

```json
{
 "id": "number (auto-increment)",
 "student_email": "string",
 "mentor_email": "string",
 "status": "paid | scheduled | completed | cancelled",
 "amount": "number (decimal)",
 "currency": "string",
 "payment_id": "string",
 "order_id": "string",
 "scheduled_at": "timestamp (optional)",
 "duration_minutes": "number (optional)",
 "meeting_link": "string (optional)",
 "notes": "text (optional)",
 "created_at": "timestamp",
 "completed_at": "timestamp (optional)"
}
...

```

#### ### Roadmap Model

```

```json
{
  "id": "number (auto-increment)",
  "owner_email": "string",
  "title": "string",
  "description": "text",
  "category": "string",
  "level": "beginner | intermediate | advanced",
  "duration": "string (e.g., '3 months')",
  "phases": "JSON (array of phase objects)",
  "tags": "string (comma-separated)",
  "created_at": "timestamp",
  "updated_at": "timestamp"
}
...

```

Auth & User

GET /api/health

****Description:**** Health check endpoint to verify API and database connectivity

****Authentication:**** None

****Response:****

```

```json
{
 "status": "ok",
 "database": "connected",
 "auth0": "configured"
}

```

...

---

#### #### GET /api/protected

**\*\*Description:\*\*** Example protected route demonstrating JWT authentication

**\*\*Authentication:\*\*** Required (JWT Bearer token)

**\*\*Headers:\*\***

...

Authorization: Bearer <jwt\_token>

...

**\*\*Response:\*\***

```json

```
{
  "message": "This is a protected route",
  "user": {
    "sub": "auth0|123456789",
    "email": "user@example.com",
    "email_verified": true
  }
}
```

```

---

#### #### POST /api/uploads/resume

**\*\*Description:\*\*** Upload resume file (PDF, DOC, or DOCX)

**\*\*Authentication:\*\*** None

**\*\*Content-Type:\*\*** `multipart/form-data`

**\*\*Request Body:\*\***

- `resume` (file): Resume file (max 10MB)

**\*\*Response:\*\***

```json

```
{
  "message": "Resume uploaded successfully",
  "url": "http://localhost:4000/uploads/resumes/1234567890_abc_resume.pdf"
}
```

```

**\*\*Error Response:\*\***

```json

```
{
  "error": "File size exceeds 10MB limit"
}
```

```

---

#### #### POST /api/users

**\*\*Description:\*\*** Store or update basic user information after Auth0 login

**\*\*Authentication:\*\*** Required (JWT Bearer token)

**\*\*Request Body:\*\***

```
```json
{
  "sub": "auth0|123456789",
  "email": "john.doe@example.com",
  "name": "John Doe",
  "picture": "https://example.com/avatar.jpg"
}
```
```

**\*\*Response:\*\***

```
```json
{
  "message": "User profile updated",
  "user": {
    "auth0_id": "auth0|123456789",
    "email": "john.doe@example.com",
    "name": "John Doe",
    "picture": "https://example.com/avatar.jpg",
    "created_at": "2026-03-04T10:30:00.000Z",
    "updated_at": "2026-03-04T10:30:00.000Z"
  }
}
```
```

---

### GET /api/users/profile

**\*\*Description:\*\*** Get user's profile information (creates from Auth0 if not exists)

**\*\*Authentication:\*\*** Required (JWT Bearer token)

**\*\*Response:\*\***

```
```json
{
  "auth0_id": "auth0|123456789",
  "email": "john.doe@example.com",
  "name": "John Doe",
  "picture": "https://example.com/avatar.jpg",
  "role": "student",
  "graduationYear": 2024,
  "course": "Computer Science",
  "linkedIn": "https://linkedin.com/in/johndoe",
  "bio": "Passionate about software development",
  "skills": "JavaScript, Python, React",
  "isOpenToMentoring": false,
  "created_at": "2026-03-04T10:30:00.000Z",
  "updated_at": "2026-03-04T10:30:00.000Z"
}
```
```

---

### PUT /api/users/profile

**\*\*Description:\*\*** Create or update user profile fields

**\*\*Authentication:\*\*** Required (JWT Bearer token)

**\*\*Request Body:\*\***

```
```json
{
```

```
"name": "John Doe",
"email": "john.doe@example.com",
"role": "student",
"graduationYear": 2024,
"course": "Computer Science",
"currentCompany": "Tech Corp",
"jobTitle": "Software Engineer",
"location": "San Francisco, CA",
"linkedIn": "https://linkedin.com/in/johndoe",
"bio": "Passionate about software development and mentoring",
"skills": "JavaScript, Python, React, Node.js",
"isOpenToMentoring": true
}
...

```

```
**Response:**
```json
{
 "message": "Profile updated successfully",
 "profile": {
 "auth0_id": "auth0|123456789",
 "email": "john.doe@example.com",
 "name": "John Doe",
 "role": "student",
 "graduationYear": 2024,
 "course": "Computer Science",
 "currentCompany": "Tech Corp",
 "jobTitle": "Software Engineer",
 "location": "San Francisco, CA",
 "linkedIn": "https://linkedin.com/in/johndoe",
 "bio": "Passionate about software development and mentoring",
 "skills": "JavaScript, Python, React, Node.js",
 "isOpenToMentoring": true,
 "updated_at": "2026-03-04T11:00:00.000Z"
 }
}
...

```

---

#### POST /api/users/sync-self

**Description:** Sync user data from Auth0 to local database

**Authentication:** Required (JWT Bearer token)

```
Response:
```json
{
  "message": "User synced successfully",
  "user": {
    "auth0_id": "auth0|123456789",
    "email": "john.doe@example.com",
    "name": "John Doe",
    "picture": "https://example.com/avatar.jpg"
  }
}
...

```

Academic Progress

GET /api/academic/semesters

****Description:**** Get all semesters for a student

****Authentication:**** None (requires auth0_id in query)

****Query Parameters:****

- `auth0\_id` (required): Student's Auth0 ID

****Request Example:****

...

GET /api/academic/semesters?auth0_id=auth0|123456789

...

****Response:****

```json

```
{
 "semesters": [
 {
 "semester_key": "sem7",
 "student_auth0_id": "auth0|123456789",
 "name": "Semester 7",
 "gpa": 8.5,
 "total_credits": 18,
 "is_current": true
 },
 {
 "semester_key": "sem6",
 "student_auth0_id": "auth0|123456789",
 "name": "Semester 6",
 "gpa": 8.2,
 "total_credits": 20,
 "is_current": false
 }
]
}
```

...

---

#### GET /api/academic/courses

**\*\*Description:\*\*** Get all courses for a specific semester

**\*\*Authentication:\*\*** None (requires auth0\_id and semester\_key in query)

**\*\*Query Parameters:\*\***

- `auth0\_id` (required): Student's Auth0 ID

- `semester\_key` (required): Semester identifier

**\*\*Request Example:\*\***

...

GET /api/academic/courses?auth0\_id=auth0|123456789&semester\_key=sem7

...

**\*\*Response:\*\***

```json

```
{
  "courses": [
    {
      "course_key": "cs501",
      "student_auth0_id": "auth0|123456789",
      "semester_key": "sem7",
```



```

    "name": "Advanced Web Development",
    "code": "CS501",
    "credits": 4,
    "grade": "A",
    "status": "completed",
    "progress": "100%"
  },
  {
    "course_key": "cs502",
    "student_auth0_id": "auth0|123456789",
    "semester_key": "sem7",
    "name": "Machine Learning",
    "code": "CS502",
    "credits": 4,
    "grade": "A-",
    "status": "in-progress",
    "progress": "75%"
  }
]
}
...

```

POST /api/academic/semesters

****Description:**** Create or update a semester (upsert by student_auth0_id + semester_key)

****Authentication:**** None

****Request Body:****

```

```json
{
 "student_auth0_id": "auth0|123456789",
 "semester_key": "sem7",
 "name": "Semester 7",
 "gpa": 8.5,
 "total_credits": 18,
 "is_current": true
}
...

```

**\*\*Response:\*\***

```

```json
{
  "message": "Semester saved successfully",
  "semester": {
    "semester_key": "sem7",
    "student_auth0_id": "auth0|123456789",
    "name": "Semester 7",
    "gpa": 8.5,
    "total_credits": 18,
    "is_current": true
  }
}
...

```

POST /api/academic/courses

****Description:**** Create or update a course (upsert by student_auth0_id + course_key)

****Authentication:**** None

****Request Body:****

```
```json
{
 "student_auth0_id": "auth0|123456789",
 "course_key": "cs501",
 "semester_key": "sem7",
 "name": "Advanced Web Development",
 "code": "CS501",
 "credits": 4,
 "grade": "A",
 "status": "completed",
 "progress": "100%"
}
```
```

****Response:****

```
```json
{
 "message": "Course saved successfully",
 "course": {
 "course_key": "cs501",
 "student_auth0_id": "auth0|123456789",
 "semester_key": "sem7",
 "name": "Advanced Web Development",
 "code": "CS501",
 "credits": 4,
 "grade": "A",
 "status": "completed",
 "progress": "100%"
 }
}
```
```

GET /api/academic-progress/me

****Description:**** Get computed academic progress with statistics (legacy endpoint)

****Authentication:**** Required (JWT Bearer token)

****Response:****

```
```json
{
 "overallGPA": 8.3,
 "totalCredits": 120,
 "completedCourses": 28,
 "inProgressCourses": 2,
 "upcomingCourses": 4,
 "semesters": [
 {
 "id": "sem7",
 "name": "Semester 7",
 "gpa": 8.5,
 "totalCredits": 18,
 "courses": [
 {
 "id": "cs501",
 "name": "Advanced Web Development",
 "code": "CS501",
 "credits": 4,

```

```
 "grade": "A",
 "status": "completed"
 }
]
}
]
}
...
```

---

#### PUT /api/academic-progress/me

**\*\*Description:\*\*** Replace user's complete academic progress (transactional)

**\*\*Authentication:\*\*** Required (JWT Bearer token)

**\*\*Request Body:\*\***

```
```json
{
  "semesters": [
    {
      "id": "sem7",
      "name": "Semester 7",
      "gpa": 8.5,
      "totalCredits": 18,
      "courses": [
        {
          "name": "Advanced Web Development",
          "code": "CS501",
          "credits": 4,
          "grade": "A",
          "status": "completed"
        }
      ]
    }
  ]
}
...
```

****Response:****

```
```json
{
 "message": "Academic progress updated successfully",
 "progress": {
 "overallGPA": 8.5,
 "totalCredits": 18,
 "completedCourses": 1
 }
}
...
```

---

## Donations

#### GET /api/donations

**\*\*Description:\*\*** List donations with pagination and optional filters

**\*\*Authentication:\*\*** None

**\*\*Query Parameters:\*\***

- `page` (optional): Page number (default: 1)
- `limit` (optional): Items per page (default: 10)
- `status` (optional): Filter by transaction status (pending|completed|failed)
- `donor\_email` (optional): Filter by donor email

**\*\*Request Example:\*\***

...

GET /api/donations?page=1&limit=10&status=completed

...

**\*\*Response:\*\***

```json

```
{
  "donations": [
    {
      "id": 1,
      "donor_name": "John Doe",
      "donor_email": "john@example.com",
      "amount": 5000.00,
      "currency": "INR",
      "payment_method": "razorpay",
      "transaction_status": "completed",
      "transaction_id": "txn_123456",
      "razorpay_payment_id": "pay_123456",
      "donation_date": "2026-03-04T10:30:00.000Z",
      "receipt_sent": true
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 45,
    "totalPages": 5
  }
}
```

...

POST /api/donations

****Description:**** Create a new donation record

****Authentication:**** None

****Request Body:****

```json

```
{
 "donor_name": "John Doe",
 "donor_email": "john@example.com",
 "amount": 5000.00,
 "currency": "INR",
 "payment_method": "razorpay",
 "razorpay_order_id": "order_123456",
 "razorpay_payment_id": "pay_123456",
 "razorpay_signature": "sig_123456",
 "notes": "Donation for scholarship fund"
}
```

...

**\*\*Response:\*\***

```json

{

```
"message": "Donation created successfully",
"donation": {
  "id": 50,
  "donor_name": "John Doe",
  "donor_email": "john@example.com",
  "amount": 5000.00,
  "currency": "INR",
  "payment_method": "razorpay",
  "transaction_status": "completed",
  "donation_date": "2026-03-04T11:00:00.000Z"
}
}
...
---
```

GET /api/donations/:id

****Description:**** Get a specific donation by ID

****Authentication:**** None

****Request Example:****

```
...
GET /api/donations/50
...
```

****Response:****

```
```json
{
 "id": 50,
 "donor_name": "John Doe",
 "donor_email": "john@example.com",
 "amount": 5000.00,
 "currency": "INR",
 "payment_method": "razorpay",
 "transaction_status": "completed",
 "transaction_id": "txn_123456",
 "razorpay_order_id": "order_123456",
 "razorpay_payment_id": "pay_123456",
 "donation_date": "2026-03-04T11:00:00.000Z",
 "receipt_sent": true,
 "notes": "Donation for scholarship fund"
}
...

```

#### PUT /api/donations/:id

**\*\*Description:\*\*** Update donation fields (status, receipt info, etc.)

**\*\*Authentication:\*\*** None

**\*\*Request Body:\*\***

```
```json
{
  "transaction_status": "completed",
  "receipt_sent": true,
  "receipt_url": "https://example.com/receipts/donation_50.pdf"
}
...
---
```

```
**Response:**
```json
{
 "message": "Donation updated successfully",
 "donation": {
 "id": 50,
 "transaction_status": "completed",
 "receipt_sent": true,
 "receipt_url": "https://example.com/receipts/donation_50.pdf"
 }
}
```
```

DELETE /api/donations/:id

****Description:**** Delete a donation (only allowed for non-completed statuses)

****Authentication:**** None

****Request Example:****

```
DELETE /api/donations/50
```
```

```
Response:
```json
{
  "message": "Donation deleted successfully"
}
```
```

```
Error Response:
```json
{
  "error": "Cannot delete completed donations"
}
```
```

---

### GET /api/donations/analytics/summary

**\*\*Description:\*\*** Get donation analytics and summary statistics

**\*\*Authentication:\*\*** None

```
Response:
```json
{
  "totalAmount": 250000.00,
  "totalDonations": 45,
  "completedDonations": 42,
  "pendingDonations": 3,
  "monthlyData": [
    {
      "month": "2026-01",
      "amount": 75000.00,
      "count": 12
    },
    {
      "month": "2026-02",

```

```

    "amount": 85000.00,
    "count": 15
  },
  {
    "month": "2026-03",
    "amount": 90000.00,
    "count": 18
  }
],
"averageDonation": 5555.56,
"topDonors": [
  {
    "donor_name": "Jane Smith",
    "donor_email": "jane@example.com",
    "totalAmount": 25000.00,
    "donationCount": 3
  }
]
}
...

```

Roadmaps

POST /api/roadmaps

****Description:**** Create a learning roadmap

****Authentication:**** None

****Request Body:****

```

```json
{
 "owner_email": "john@example.com",
 "title": "Full Stack Web Development Roadmap",
 "description": "Complete roadmap to become a full-stack developer",
 "category": "Web Development",
 "level": "intermediate",
 "duration": "6 months",
 "phases": [
 {
 "title": "Frontend Fundamentals",
 "duration": "2 months",
 "topics": ["HTML", "CSS", "JavaScript", "React"]
 },
 {
 "title": "Backend Development",
 "duration": "2 months",
 "topics": ["Node.js", "Express", "PostgreSQL"]
 },
 {
 "title": "DevOps & Deployment",
 "duration": "2 months",
 "topics": ["Docker", "CI/CD", "AWS"]
 }
],
 "tags": "javascript, react, node.js, full-stack"
}
...

```

**\*\*Response:\*\***

```

```json

```

```
{
  "message": "Roadmap created successfully",
  "roadmap": {
    "id": 10,
    "owner_email": "john@example.com",
    "title": "Full Stack Web Development Roadmap",
    "category": "Web Development",
    "level": "intermediate",
    "duration": "6 months",
    "created_at": "2026-03-04T11:00:00.000Z"
  }
}
...
---
```

GET /api/roadmaps

Description: List roadmaps with optional filtering and pagination

Authentication: None

Query Parameters:

- `owner\_email` (optional): Filter by creator email
- `category` (optional): Filter by category
- `level` (optional): Filter by level (beginner|intermediate|advanced)
- `page` (optional): Page number
- `limit` (optional): Items per page

Request Example:

```
GET /api/roadmaps?category=Web%20Development&level=intermediate&page=1
...
```

Response:

```
...json
{
  "roadmaps": [
    {
      "id": 10,
      "owner_email": "john@example.com",
      "title": "Full Stack Web Development Roadmap",
      "description": "Complete roadmap to become a full-stack developer",
      "category": "Web Development",
      "level": "intermediate",
      "duration": "6 months",
      "tags": "javascript, react, node.js, full-stack",
      "created_at": "2026-03-04T11:00:00.000Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 25
  }
}
...
```

GET /api/roadmaps/:id

Description: Get detailed roadmap by ID

****Authentication:**** None

****Request Example:****

...

GET /api/roadmaps/10

...

****Response:****

```json

```
{
 "id": 10,
 "owner_email": "john@example.com",
 "title": "Full Stack Web Development Roadmap",
 "description": "Complete roadmap to become a full-stack developer",
 "category": "Web Development",
 "level": "intermediate",
 "duration": "6 months",
 "phases": [
 {
 "title": "Frontend Fundamentals",
 "duration": "2 months",
 "topics": ["HTML", "CSS", "JavaScript", "React"]
 },
 {
 "title": "Backend Development",
 "duration": "2 months",
 "topics": ["Node.js", "Express", "PostgreSQL"]
 }
],
 "tags": "javascript, react, node.js, full-stack",
 "created_at": "2026-03-04T11:00:00.000Z",
 "updated_at": "2026-03-04T11:00:00.000Z"
}
```

...

---

#### PUT /api/roadmaps/:id

**\*\*Description:\*\*** Update roadmap fields

**\*\*Authentication:\*\*** None

**\*\*Request Body:\*\***

```json

```
{
  "title": "Updated Full Stack Roadmap",
  "description": "Enhanced roadmap with more details",
  "duration": "8 months"
}
```

...

****Response:****

```json

```
{
 "message": "Roadmap updated successfully",
 "roadmap": {
 "id": 10,
 "title": "Updated Full Stack Roadmap",
 "updated_at": "2026-03-04T12:00:00.000Z"
 }
}
```

...

---

#### DELETE /api/roadmaps/:id

**Description:** Delete a roadmap

**Authentication:** None

**Request Example:**

...

DELETE /api/roadmaps/10

...

**Response:**

```json

```
{
  "message": "Roadmap deleted successfully"
}
```

...

Jobs & Applications

POST /api/jobs

Description: Create a new job or internship posting

Authentication: None

Request Body:

```json

```
{
 "title": "Full Stack Developer",
 "company": "Tech Corp",
 "description": "We are looking for an experienced full-stack developer...",
 "requirements": "3+ years of experience with React and Node.js",
 "job_type": "full-time",
 "industry": "Technology",
 "location": "San Francisco, CA",
 "remote": true,
 "salary_min": 80000,
 "salary_max": 120000,
 "application_deadline": "2026-04-30",
 "posted_by": "recruiter@techcorp.com",
 "status": "active"
}
```

...

**Response:**

```json

```
{
  "message": "Job created successfully",
  "job": {
    "id": 25,
    "title": "Full Stack Developer",
    "company": "Tech Corp",
    "job_type": "full-time",
    "location": "San Francisco, CA",
    "remote": true,
    "status": "active",
    "created_at": "2026-03-04T11:00:00.000Z"
  }
}
```

```
}  
...
```

GET /api/jobs

Description: List jobs with filtering and pagination

Authentication: None

Query Parameters:

- `q` (optional): Search query (title, company, description)
- `industry` (optional): Filter by industry
- `job\_type` (optional): full-time|part-time|internship|contract
- `location` (optional): Filter by location
- `remote\_only` (optional): true|false
- `status` (optional): active|closed|draft
- `posted\_by` (optional): Filter by poster email
- `page` (optional): Page number
- `limit` (optional): Items per page

Request Example:

...

GET /api/jobs?q=developer&job_type=full-time&remote_only=true&page=1&limit=10

...

Response:

```json

```
{
 "jobs": [
 {
 "id": 25,
 "title": "Full Stack Developer",
 "company": "Tech Corp",
 "description": "We are looking for an experienced full-stack developer...",
 "job_type": "full-time",
 "industry": "Technology",
 "location": "San Francisco, CA",
 "remote": true,
 "salary_min": 80000,
 "salary_max": 120000,
 "application_deadline": "2026-04-30",
 "posted_by": "recruiter@techcorp.com",
 "status": "active",
 "created_at": "2026-03-04T11:00:00.000Z"
 },
 ...
],
 "pagination": {
 "page": 1,
 "limit": 10,
 "total": 45,
 "totalPages": 5
 }
}
```

...

---

### GET /api/jobs/:id

**Description:** Get detailed job posting by ID

**\*\*Authentication:\*\*** None

**\*\*Request Example:\*\***

...

GET /api/jobs/25

...

**\*\*Response:\*\***

```json

```
{
  "id": 25,
  "title": "Full Stack Developer",
  "company": "Tech Corp",
  "description": "We are looking for an experienced full-stack developer...",
  "requirements": "3+ years of experience with React and Node.js",
  "job_type": "full-time",
  "industry": "Technology",
  "location": "San Francisco, CA",
  "remote": true,
  "salary_min": 80000,
  "salary_max": 120000,
  "application_deadline": "2026-04-30",
  "posted_by": "recruiter@techcorp.com",
  "status": "active",
  "created_at": "2026-03-04T11:00:00.000Z",
  "updated_at": "2026-03-04T11:00:00.000Z"
}
```

...

PUT /api/jobs/:id

****Description:**** Update job posting fields

****Authentication:**** None

****Request Body:****

```json

```
{
 "title": "Senior Full Stack Developer",
 "salary_min": 90000,
 "salary_max": 140000,
 "status": "active"
}
```

...

**\*\*Response:\*\***

```json

```
{
  "message": "Job updated successfully",
  "job": {
    "id": 25,
    "title": "Senior Full Stack Developer",
    "salary_min": 90000,
    "salary_max": 140000,
    "updated_at": "2026-03-04T12:00:00.000Z"
  }
}
```

...

DELETE /api/jobs/:id

Description: Delete a job posting

Authentication: None

Request Example:

```
...
DELETE /api/jobs/25
...
```

Response:

```
...json
{
  "message": "Job deleted successfully"
}
...
```

POST /api/jobs/:id/apply

Description: Submit a job application (student role required)

Authentication: None (requires student role verification via email)

Request Body:

```
...json
{
  "applicant_email": "student@example.com",
  "resume_url": "http://localhost:4000/uploads/resumes/student_resume.pdf",
  "cover_letter": "I am very interested in this position because..."
}
...
```

Response:

```
...json
{
  "message": "Application submitted successfully",
  "application": {
    "id": 100,
    "job_id": 25,
    "applicant_email": "student@example.com",
    "status": "applied",
    "applied_at": "2026-03-04T11:30:00.000Z"
  }
}
...
```

Error Response:

```
...json
{
  "error": "Only students can apply to jobs"
}
...
```

GET /api/applications

Description: List applications submitted by a student

Authentication: None

****Query Parameters:****

- `applicant\_email` (required): Student's email

****Request Example:****

...

GET /api/applications?applicant_email=student@example.com

...

****Response:****

```json

```
{
 "applications": [
 {
 "id": 100,
 "job_id": 25,
 "job_title": "Full Stack Developer",
 "company": "Tech Corp",
 "applicant_email": "student@example.com",
 "resume_url": "http://localhost:4000/uploads/resumes/student_resume.pdf",
 "cover_letter": "I am very interested in this position...",
 "status": "applied",
 "applied_at": "2026-03-04T11:30:00.000Z",
 "updated_at": "2026-03-04T11:30:00.000Z"
 }
]
}
```

...

---

#### GET /api/applications/by-job

**\*\*Description:\*\*** List all applicants for a specific job

**\*\*Authentication:\*\*** None

**\*\*Query Parameters:\*\***

- `job\_id` (required): Job ID

**\*\*Request Example:\*\***

...

GET /api/applications/by-job?job\_id=25

...

**\*\*Response:\*\***

```json

```
{
  "job": {
    "id": 25,
    "title": "Full Stack Developer",
    "company": "Tech Corp"
  },
  "applications": [
    {
      "id": 100,
      "applicant_email": "student@example.com",
      "applicant_name": "John Student",
      "resume_url": "http://localhost:4000/uploads/resumes/student_resume.pdf",
      "status": "applied",
      "applied_at": "2026-03-04T11:30:00.000Z"
    }
  ],
  "total": 15
}
```

```
}  
...
```

PUT /api/applications/:id

Description: Update application status

Authentication: None

Request Body:

```
```json  
{
 "status": "accepted"
}
...
```

**Response:**

```
```json  
{  
  "message": "Application status updated",  
  "application": {  
    "id": 100,  
    "status": "accepted",  
    "updated_at": "2026-03-04T12:00:00.000Z"  
  }  
}  
...
```

Valid Status Values: `applied`, `withdrawn`, `accepted`, `rejected`

Messaging

POST /api/messages

Description: Send an encrypted message between two users (mentorship chat gating applies)

Authentication: None

Request Body:

```
```json  
{
 "sender_email": "student@example.com",
 "receiver_email": "mentor@example.com",
 "content": "Hi! I would like to discuss career opportunities in web development."
}
...
```

**Response:**

```
```json  
{  
  "message": "Message sent successfully",  
  "messageData": {  
    "id": 150,  
    "sender_email": "student@example.com",  
    "receiver_email": "mentor@example.com",  
    "is_read": false,  
    "created_at": "2026-03-04T11:30:00.000Z"  
  }  
}
```

...

****Note:**** Content is automatically encrypted before storage. Mentorship conversations have a free message limit before requiring a subscription.

GET /api/messages

****Description:**** Get messages in a conversation thread (automatically decrypted)

****Authentication:**** None

****Query Parameters:****

- `user` (required): Current user's email
- `with` (required): Other user's email

****Request Example:****

...

GET /api/messages?user=student@example.com&with=mentor@example.com

...

****Response:****

```json

```
{
 "messages": [
 {
 "id": 150,
 "sender_email": "student@example.com",
 "receiver_email": "mentor@example.com",
 "content": "Hi! I would like to discuss career opportunities in web development.",
 "is_read": true,
 "created_at": "2026-03-04T11:30:00.000Z"
 },
 {
 "id": 151,
 "sender_email": "mentor@example.com",
 "receiver_email": "student@example.com",
 "content": "Hello! I'd be happy to help. Let's schedule a session.",
 "is_read": true,
 "created_at": "2026-03-04T11:45:00.000Z"
 }
],
 "total": 2
}
```

...

---

#### ### GET /api/messages/threads

**\*\*Description:\*\*** List all message threads for a user with latest message

**\*\*Authentication:\*\*** None

**\*\*Query Parameters:\*\***

- `user\_email` (required): User's email

**\*\*Request Example:\*\***

...

GET /api/messages/threads?user\_email=student@example.com

...

**\*\*Response:\*\***



```

```json
{
  "threads": [
    {
      "other_user_email": "mentor@example.com",
      "other_user_name": "Jane Mentor",
      "other_user_picture": "https://example.com/avatar.jpg",
      "latest_message": {
        "content": "Hello! I'd be happy to help.",
        "created_at": "2026-03-04T11:45:00.000Z",
        "is_read": true,
        "sender_email": "mentor@example.com"
      },
      "unread_count": 0
    }
  ]
}
```

```

---

#### POST /api/messages/connected

**Description:** Send message only if users are connected (enforces connection requirement)

**Authentication:** None

**Request Body:**

```

```json
{
  "sender_email": "student@example.com",
  "receiver_email": "alumni@example.com",
  "content": "Thank you for accepting my connection request!"
}
```

```

**Response:**

```

```json
{
  "message": "Message sent successfully",
  "messageData": {
    "id": 152,
    "sender_email": "student@example.com",
    "receiver_email": "alumni@example.com",
    "created_at": "2026-03-04T12:00:00.000Z"
  }
}
```

```

**Error Response:**

```

```json
{
  "error": "Users must be connected to send messages"
}
```

```

---

## Connections (network)

#### POST /api/connections/request

**Description:** Send a connection request to another user

**\*\*Authentication:\*\*** None

**\*\*Request Body:\*\***

```
```json
{
  "requester_email": "student@example.com",
  "target_email": "alumni@example.com",
  "message": "Hi! I'm interested in learning about your career path in software engineering."
}
```
```

**\*\*Response:\*\***

```
```json
{
  "message": "Connection request sent successfully",
  "connection": {
    "id": 50,
    "requester_email": "student@example.com",
    "target_email": "alumni@example.com",
    "status": "pending",
    "created_at": "2026-03-04T11:00:00.000Z"
  }
}
```
```

---

#### POST /api/connections/respond

**\*\*Description:\*\*** Accept or reject a connection request

**\*\*Authentication:\*\*** None

**\*\*Request Body:\*\***

```
```json
{
  "user_email": "alumni@example.com",
  "other_email": "student@example.com",
  "action": "accept"
}
```
```

**\*\*Response:\*\***

```
```json
{
  "message": "Connection request accepted",
  "connection": {
    "id": 50,
    "requester_email": "student@example.com",
    "target_email": "alumni@example.com",
    "status": "accepted",
    "responded_at": "2026-03-04T11:15:00.000Z"
  }
}
```
```

**\*\*Valid Actions:\*\*** `accept`, `reject`

---

#### GET /api/connections

**\*\*Description:\*\*** List connections for a user

**\*\*Authentication:\*\*** None

**\*\*Query Parameters:\*\***

- `user\_email` (required): User's email
- `status` (optional): Filter by status (pending|accepted|rejected|removed)

**\*\*Request Example:\*\***

...

GET /api/connections?user\_email=student@example.com&status=accepted

...

**\*\*Response:\*\***

```json

```
{
  "connections": [
    {
      "id": 50,
      "requester_email": "student@example.com",
      "target_email": "alumni@example.com",
      "status": "accepted",
      "message": "Hi! I'm interested in learning about your career path...",
      "created_at": "2026-03-04T11:00:00.000Z",
      "responded_at": "2026-03-04T11:15:00.000Z",
      "other_user": {
        "email": "alumni@example.com",
        "name": "Jane Alumni",
        "picture": "https://example.com/avatar.jpg",
        "currentCompany": "Tech Corp",
        "jobTitle": "Senior Engineer"
      }
    }
  ],
  "total": 15,
  "pending": 2,
  "accepted": 13
}
```

...

POST /api/connections/remove

****Description:**** Remove a connection (sets status to 'removed')

****Authentication:**** None

****Request Body:****

```json

```
{
 "user_email": "student@example.com",
 "other_email": "alumni@example.com"
}
```

...

**\*\*Response:\*\***

```json

```
{
  "message": "Connection removed successfully",
  "connection": {
    "id": 50,
    "status": "removed"
  }
}
```

...

Mentorship (mentors, requests, sessions, ratings)

GET /api/mentors

Description: List available mentors with filtering

Authentication: None

Query Parameters:

- `q` (optional): Search query (name, expertise)
- `min\_experience` (optional): Minimum years of experience
- `max\_price` (optional): Maximum hourly rate
- `min\_rating` (optional): Minimum rating (1-5)
- `page` (optional): Page number
- `limit` (optional): Items per page

Request Example:

...

GET /api/mentors?q=web%20development&min_experience=5&max_price=100&page=1

...

Response:

```json

```
{
 "mentors": [
 {
 "mentor_email": "mentor@example.com",
 "name": "Jane Mentor",
 "picture": "https://example.com/avatar.jpg",
 "expertise_areas": "Web Development, React, Node.js",
 "years_experience": 8,
 "hourly_rate": 75.00,
 "availability": "Weekday evenings",
 "bio": "Experienced full-stack developer passionate about mentoring...",
 "rating": 4.8,
 "total_sessions": 45,
 "total_ratings": 42,
 "currentCompany": "Tech Corp",
 "jobTitle": "Senior Engineer"
 }
],
 "pagination": {
 "page": 1,
 "limit": 10,
 "total": 25
 }
}
```

...

---

### GET /api/mentors/profile

**Description:** Get a specific mentor's profile

**Authentication:** None

**Query Parameters:**

- `email` (required): Mentor's email

**\*\*Request Example:\*\***

...

GET /api/mentors/profile?email=mentor@example.com

...

**\*\*Response:\*\***

```json

```
{
  "mentor_email": "mentor@example.com",
  "name": "Jane Mentor",
  "picture": "https://example.com/avatar.jpg",
  "expertise_areas": "Web Development, React, Node.js",
  "years_experience": 8,
  "hourly_rate": 75.00,
  "availability": "Weekday evenings",
  "bio": "Experienced full-stack developer passionate about mentoring...",
  "rating": 4.8,
  "total_sessions": 45,
  "total_ratings": 42,
  "currentCompany": "Tech Corp",
  "jobTitle": "Senior Engineer",
  "linkedIn": "https://linkedin.com/in/janementor",
  "created_at": "2026-01-10T10:00:00.000Z",
  "updated_at": "2026-03-04T09:00:00.000Z"
}
```

...

POST /api/mentors/profile

****Description:**** Create or update mentor profile (for alumni)

****Authentication:**** None

****Request Body:****

```json

```
{
 "mentor_email": "mentor@example.com",
 "expertise_areas": "Web Development, React, Node.js, AWS",
 "years_experience": 8,
 "hourly_rate": 75.00,
 "availability": "Weekday evenings and weekends",
 "bio": "Experienced full-stack developer passionate about mentoring the next generation of engineers."
}
```

...

**\*\*Response:\*\***

```json

```
{
  "message": "Mentor profile updated successfully",
  "profile": {
    "mentor_email": "mentor@example.com",
    "expertise_areas": "Web Development, React, Node.js, AWS",
    "years_experience": 8,
    "hourly_rate": 75.00,
    "updated_at": "2026-03-04T11:00:00.000Z"
  }
}
```

...

POST /api/mentorship/request

Description: Student requests mentorship from a mentor

Authentication: None

Request Body:

```
```json
{
 "student_email": "student@example.com",
 "mentor_email": "mentor@example.com",
 "message": "I'm interested in learning about web development career paths and would love your guidance."
}
```

**Response:**

```
```json
{
  "message": "Mentorship request sent successfully",
  "request": {
    "id": 75,
    "student_email": "student@example.com",
    "mentor_email": "mentor@example.com",
    "status": "pending",
    "created_at": "2026-03-04T11:00:00.000Z"
  }
}
```

POST /api/mentorship/respond

Description: Mentor responds to mentorship request

Authentication: None

Request Body:

```
```json
{
 "mentor_email": "mentor@example.com",
 "student_email": "student@example.com",
 "action": "accept"
}
```

**Response:**

```
```json
{
  "message": "Mentorship request accepted",
  "request": {
    "id": 75,
    "status": "accepted",
    "responded_at": "2026-03-04T11:15:00.000Z"
  }
}
```

Valid Actions: `accept`, `reject`

GET /api/mentorship/requests

****Description:**** List mentorship requests for mentor or student

****Authentication:**** None

****Query Parameters:****

- `mentor\_email` or `student\_email` (one required): Filter by user
- `status` (optional): Filter by status (pending|accepted|rejected)

****Request Example:****

GET /api/mentorship/requests?student_email=student@example.com&status=accepted

****Response:****

```json

```
{
 "requests": [
 {
 "id": 75,
 "student_email": "student@example.com",
 "mentor_email": "mentor@example.com",
 "mentor_name": "Jane Mentor",
 "mentor_picture": "https://example.com/avatar.jpg",
 "message": "I'm interested in learning about web development...",
 "status": "accepted",
 "created_at": "2026-03-04T11:00:00.000Z",
 "responded_at": "2026-03-04T11:15:00.000Z"
 }
],
 "total": 5,
 "pending": 1,
 "accepted": 4
}
```

---

---

### POST /api/mentorship/sessions/purchase

**\*\*Description:\*\*** Create a paid session record after successful payment

**\*\*Authentication:\*\*** None

**\*\*Request Body:\*\***

```json

```
{
  "student_email": "student@example.com",
  "mentor_email": "mentor@example.com",
  "amount": 75.00,
  "currency": "USD",
  "payment_id": "pay_123456",
  "order_id": "order_123456"
}
```

****Response:****

```json

```
{
 "message": "Session purchased successfully",
 "session": {
 "id": 100,
 "student_email": "student@example.com",
 "mentor_email": "mentor@example.com",

```

```
 "status": "paid",
 "amount": 75.00,
 "currency": "USD",
 "payment_id": "pay_123456",
 "created_at": "2026-03-04T11:30:00.000Z"
 }
}
...

```

### POST /api/mentorship/sessions/schedule

**Description:** Schedule a paid session with meeting details

**Authentication:** None

**Request Body:**

```
```json
{
  "session_id": 100,
  "scheduled_at": "2026-03-10T14:00:00.000Z",
  "duration_minutes": 60,
  "meeting_link": "https://meet.google.com/abc-defg-hij",
  "notes": "Looking forward to discussing career paths in web development"
}
...
---
```

Response:

```
```json
{
 "message": "Session scheduled successfully",
 "session": {
 "id": 100,
 "status": "scheduled",
 "scheduled_at": "2026-03-10T14:00:00.000Z",
 "duration_minutes": 60,
 "meeting_link": "https://meet.google.com/abc-defg-hij"
 }
}
...

```

### POST /api/mentorship/sessions/complete

**Description:** Mark session as completed (mentor action)

**Authentication:** None

**Request Body:**

```
```json
{
  "session_id": 100
}
...
---
```

Response:

```
```json
{
 "message": "Session marked as completed",
 "session": {
 "id": 100,
```



```
 "status": "completed",
 "completed_at": "2026-03-10T15:00:00.000Z"
 }
}
...

```

#### ### GET /api/mentorship/sessions

**Description:** List sessions for mentor or student

**Authentication:** None

**Query Parameters:**

- `mentor\_email` or `student\_email` (one required): Filter by user
- `status` (optional): Filter by status (paid|scheduled|completed|cancelled)

**Request Example:**

```
GET /api/mentorship/sessions?student_email=student@example.com&status=scheduled
...

```

**Response:**

```
```json
{
  "sessions": [
    {
      "id": 100,
      "student_email": "student@example.com",
      "mentor_email": "mentor@example.com",
      "mentor_name": "Jane Mentor",
      "status": "scheduled",
      "amount": 75.00,
      "currency": "USD",
      "scheduled_at": "2026-03-10T14:00:00.000Z",
      "duration_minutes": 60,
      "meeting_link": "https://meet.google.com/abc-defg-hij",
      "notes": "Looking forward to discussing career paths",
      "created_at": "2026-03-04T11:30:00.000Z"
    }
  ],
  "total": 3,
  "scheduled": 1,
  "completed": 2
}
...
---
```

POST /api/mentorship/ratings

Description: Submit rating and feedback after completed session (updates mentor's aggregate rating)

Authentication: None

Request Body:

```
```json
{
 "session_id": 100,
 "student_email": "student@example.com",
 "mentor_email": "mentor@example.com",
 "rating": 5,

```

```
"feedback": "Excellent session! Very insightful guidance on career paths and technical skills."
}
...
```

```
Response:
```json
{
  "message": "Rating submitted successfully",
  "rating": {
    "id": 50,
    "session_id": 100,
    "rating": 5,
    "feedback": "Excellent session! Very insightful guidance...",
    "created_at": "2026-03-10T15:30:00.000Z"
  },
  "mentor_updated": {
    "mentor_email": "mentor@example.com",
    "rating": 4.85,
    "total_ratings": 43
  }
}
...
```

****Valid Rating Values:**** 1-5 (integer)

Frontend Next.js API Routes (proxies & payment handlers)

These are serverless API routes implemented under `frontend/src/app/api/\*` that either proxy to the backend or handle payments

GET /api/mentors/profile (Next.js Proxy)

****File:**** `frontend/src/app/api/mentors/profile/route.ts`

****Description:**** Proxies to backend mentor profile endpoint

****Authentication:**** None for GET; server-side token for POST

****Query Parameters:****

- `email` (required): Mentor's email

****Request Example:****

...

GET /api/mentors/profile?email=mentor@example.com

...

****Response:****

```
```json
{
 "mentor_email": "mentor@example.com",
 "name": "Jane Mentor",
 "expertise_areas": "Web Development, React, Node.js",
 "years_experience": 8,
 "hourly_rate": 75.00,
 "rating": 4.8
}
...
```

---

#### POST /api/payment/create-order

\*\*File:\*\* `frontend/src/app/api/payment/create-order/route.ts`

\*\*Description:\*\* Create Razorpay payment order

\*\*Authentication:\*\* None

\*\*Environment Variables Required:\*\*

- `RAZORPAY\_KEY\_ID`
- `RAZORPAY\_KEY\_SECRET`

\*\*Request Body:\*\*

```
```json
{
  "amount": 7500,
  "currency": "INR",
  "receipt": "receipt_order_100",
  "notes": {
    "session_id": "100",
    "student_email": "student@example.com",
    "mentor_email": "mentor@example.com"
  }
}
```
```

\*\*Response:\*\*

```
```json
{
  "orderId": "order_123456",
  "amount": 7500,
  "currency": "INR",
  "key": "rzp_test_xxxxxxxxxxxxx"
}
```
```

---

#### POST /api/payment/verify

\*\*File:\*\* `frontend/src/app/api/payment/verify/route.ts`

\*\*Description:\*\* Verify Razorpay payment signature and store donation

\*\*Authentication:\*\* None

\*\*Request Body:\*\*

```
```json
{
  "paymentId": "pay_123456",
  "orderId": "order_123456",
  "signature": "generated_signature_string",
  "donationData": {
    "donor_name": "John Doe",
    "donor_email": "john@example.com",
    "amount": 5000.00,
    "currency": "INR",
    "payment_method": "razorpay",
    "notes": "Scholarship fund donation"
  }
}
```
```

**\*\*Response:\*\***

```
```json
{
  "success": true,
  "message": "Payment verified and donation recorded",
  "donation": {
    "id": 75,
    "transaction_status": "completed",
    "razorpay_payment_id": "pay_123456"
  }
}
```
```

**\*\*Error Response:\*\***

```
```json
{
  "success": false,
  "error": "Payment verification failed",
  "details": "Invalid signature"
}
```
```

---

#### POST /api/payment/webhook

**\*\*File:\*\*** `frontend/src/app/api/payment/webhook/route.ts`

**\*\*Description:\*\*** Razorpay webhook receiver for payment events

**\*\*Authentication:\*\*** Webhook signature verification via `x-razorpay-signature` header

**\*\*Headers:\*\***

```
```
x-razorpay-signature: generated_webhook_signature
```
```

**\*\*Request Body (example payment.captured event):\*\***

```
```json
{
  "event": "payment.captured",
  "payload": {
    "payment": {
      "entity": {
        "id": "pay_123456",
        "amount": 750000,
        "currency": "INR",
        "status": "captured",
        "order_id": "order_123456",
        "email": "john@example.com"
      }
    }
  }
}
```
```

**\*\*Response:\*\***

```
```json
{
  "status": "received",
  "event": "payment.captured"
}
```
```

---

### ### Donations API Routes (Frontend Serverless)

**\*\*Files:\*\*** `frontend/src/app/api/donations/\*`

**\*\*Description:\*\*** Complete CRUD operations for donations using direct database connection

##### GET /api/donations

List donations with pagination (same as backend endpoint)

##### POST /api/donations

Create donation record (same as backend endpoint)

##### GET /api/donations/:id

Get specific donation (same as backend endpoint)

##### PUT /api/donations

Update donation via body (slightly different from backend PUT /:id)

##### DELETE /api/donations/:id

Delete donation (same as backend endpoint)

##### GET /api/donations/donor

Get donations by donor email

**\*\*Query Parameters:\*\***

- `email` (required): Donor's email

##### GET /api/donations/analytics

Get donation analytics and summary (same as backend endpoint)

**\*\*Note:\*\*** These routes implement the same functionality as the backend donation endpoints but run serverless within Next.js.

---

### ### POST /api/user/register (Next.js)

**\*\*File:\*\*** `frontend/src/app/api/user/register/route.ts`

**\*\*Description:\*\*** Register user profile via server-side Auth0 token

**\*\*Authentication:\*\*** Server-side (uses Auth0 Pages API)

**\*\*Request Body:\*\***

```
```json
{
  "email": "john@example.com",
  "name": "John Doe",
  "role": "student",
  "graduationYear": 2024,
  "course": "Computer Science"
}
```

****Response:****

```
```json
{
 "success": true,
 "message": "Profile created successfully",
 "profile": {
 "email": "john@example.com",
 "name": "John Doe",
```

```
 "role": "student"
 }
}
```

---

### GET /api/user/profile (Next.js)

\*\*File:\*\* `frontend/src/app/api/user/profile/route.ts`

\*\*Description:\*\* Get user profile with new user flag

\*\*Authentication:\*\* Server-side (uses Auth0 Pages API)

\*\*Response:\*\*

```
```json
{
  "profile": {
    "auth0_id": "auth0|123456789",
    "email": "john@example.com",
    "name": "John Doe",
    "role": "student",
    "graduationYear": 2024
  },
  "isNewUser": false
}
```

Additional Resources

Authentication Flow

1. User logs in via Auth0
2. Frontend receives JWT token
3. Token stored in context via `useAuthToken` hook
4. API requests include `Authorization: Bearer <token>` header
5. Backend validates token using `checkJwt` middleware

Error Handling

All endpoints follow consistent error response format:

```
```json
{
 "error": "Error message describing what went wrong",
 "details": "Additional context (optional)",
 "code": "ERROR_CODE (optional)"
}
```

Common HTTP status codes:

- `200`: Success
- `201`: Created
- `400`: Bad Request (validation error)
- `401`: Unauthorized (missing/invalid token)
- `403`: Forbidden (insufficient permissions)
- `404`: Not Found
- `500`: Internal Server Error

### Rate Limiting

- Mentorship messaging: Free tier limited to N messages, then requires paid session
- File uploads: Max 10MB per file
- API calls: No enforced rate limit currently (consider implementing)

### ### Database Schema

See `backend/database/` folder for database schema definitions and setup scripts:

- `academicProgress.js`: Academic semesters and courses tables
- `applications.js`: Job applications schema
- `connections.js`: User connections/network schema
- `donations.js`: Donations schema
- `events.js`: Events schema
- `jobs.js`: Jobs/internships schema
- `memories.js`: Memories/posts schema
- `mentorship.js`: Mentorship profiles, requests, sessions, ratings
- `messages.js`: Encrypted messaging schema
- `roadmaps.js`: Learning roadmaps schema
- `setup.sql`: Initial database setup

---

### ## Changelog

**\*\*March 4, 2026\*\***

- Enhanced documentation with detailed request/response examples
- Added Data Models section with complete schema definitions
- Organized endpoints by category with table of contents
- Added error handling and authentication flow documentation

**\*\*February 5, 2026\*\***

- Initial API documentation created
- Listed all backend Express endpoints
- Documented frontend Next.js API routes
- Added payment integration endpoints

---

**\*\*Documentation Generated:\*\* March 4, 2026**

**\*\*API Version:\*\* 1.0**

**\*\*Backend Framework:\*\* Express.js (Node.js)**

**\*\*Frontend Framework:\*\* Next.js 15 (App Router)**

**\*\*Database:\*\* PostgreSQL (Neon + local)**

**\*\*Payment Gateway:\*\* Razorpay**